K.R. MANGALAM UNIVERSITY

ENSP465 – CAPSTONE PROJECT

PHASE I REPORT

Project Title: Smart Expense Tracker Using Go

Submitted By:

Shahbaz Ali (Roll Number: 2401830007)

Harshit (Roll Number: 2401830008)

Submitted To: Dr. Hemraj Kumawat

Programme: B.Sc. Cyber Security

Semester: V

1. Introduction and Background

1.1 Problem Statement

Managing daily expenses is a common problem for students and other people who want to keep track of their money. Small expenses, such as travelling, buying food, shopping and paying for subscriptions, may not seem important individually. However, when these expenses are added together, they can have a significant effect on a person's monthly budget.

Many people depend on memory or informal notes to remember where their money has been spent. This approach is not always reliable because some transactions may be forgotten, and it can be difficult to calculate total spending at the end of the month. Using a notebook or a basic spreadsheet is another option, but it may require manual calculations and regular updates.

A simple digital system can make this process easier. It should allow users to record their income and expenses, organise transactions into categories and review their spending whenever required.

The proposed project, **Smart Expense Tracker Using Go**, aims to address this problem by designing a small application for recording and managing personal financial transactions. The main focus of Phase I is to identify the problem, define the requirements and prepare the system design before implementation.

1.2 Motivation

The main motivation behind this project is to develop a practical application that solves an everyday problem while providing an opportunity to learn software development using Go.

Students often have a limited budget and need to manage expenses related to food, transport, college activities and personal purchases. Keeping a record of these expenses can help them understand their spending habits and plan their money more carefully.

Another reason for selecting this project is its manageable scope. It involves important programming concepts such as data handling, input validation, functions, error handling and database operations. These concepts can be explored through a useful application rather than only through isolated programming exercises.

The project also provides an opportunity to consider basic security practices. Since expense records contain personal financial information, the design should consider safe data storage, input validation and appropriate access to stored information.

The goal is not to build a complicated financial platform. Instead, the project aims to create a straightforward and understandable expense-tracking application that can be expanded if required.

1.3 Existing Problem and Need

Manual expense tracking has several limitations:

- Transactions may be forgotten or recorded incorrectly.
- Calculating total expenses can take time.
- Finding transactions from a particular category or period may be inconvenient.
- Users may not have a clear picture of their overall spending.
- Poor input handling can lead to incorrect or inconsistent records.

The proposed system will address these issues by maintaining organised transaction records and providing basic summaries of income and expenses.

2. Proposed Solution and Justification for Using Go

2.1 Proposed Solution

The proposed system is a personal expense tracker developed using the Go programming language. It will allow a user to record financial transactions and review them in an organised format.

Each transaction will contain details such as the transaction type, amount, category, date and an optional description. The transaction type will indicate whether the entry represents income or an expense.

The system will validate user input before saving a transaction. For example, an amount must be valid and greater than zero, and required fields must not be left empty. This will help reduce errors in the stored data.

The application is planned to provide the following features:

1. **Add a transaction:** Record a new income or expense entry.
2. **View transactions:** Display previously recorded entries.
3. **Categorise transactions:** Group expenses into categories such as food, transport, education and shopping.
4. **Calculate totals:** Show total income, total expenses and the remaining balance.
5. **Review spending:** Generate basic summaries to help users understand their expenses.
6. **Manage records:** Provide options to update or delete an entry where supported by the final design.
7. **Handle errors:** Display useful messages when input is invalid or an operation cannot be completed.

The final interface and storage method will be selected according to the project requirements and implementation constraints. The design will remain focused on the core expense-tracking functions.

2.2 Why Go?

Go, also known as Golang, is the primary programming language selected for this project. It is a statically typed, compiled language designed to support simple and efficient software development.

One reason for choosing Go is its relatively clear syntax. Its structure makes it possible to divide an application into smaller functions and packages, which can make the code easier to understand, test and maintain.

Go also provides built-in support for error handling. This is useful in an expense tracker because the application needs to deal with invalid amounts, missing information and possible storage errors without behaving unpredictably.

Another advantage is its standard library. Packages for input/output, data encoding, testing and other common operations can reduce the need for additional dependencies. Go can also work with databases through suitable database drivers and interfaces.

From a security perspective, Go provides useful features for building applications with structured input handling and explicit error checks. However, choosing a programming language does not

automatically make an application secure. The implementation must still validate input, handle errors carefully and protect stored information.

Go is therefore suitable for this project because it offers a practical balance of readability, performance, maintainability and useful built-in functionality.

2.3 Proposed Technology Stack

The proposed technology stack is outlined below.

Component	Proposed Technology
Programming Language	Go
Application Interface	Command-line interface for the initial version
Data Storage	SQLite or another suitable local storage option
Data Handling	Go standard library and database driver, where required
Testing	Go testing package
Development Environment	Go-supported editor or IDE

A command-line interface is a reasonable starting point because it keeps the initial design simple and allows the main functions to be developed without the additional complexity of a graphical interface.

The storage choice will be finalised during implementation based on the requirements. Regardless of the selected option, the application should preserve transaction records between sessions.

3. Objectives and Scope

3.1 Objectives

The primary objective is to design and develop a personal expense tracker using Go that can record transactions and present basic financial summaries.

The specific objectives are:

- To provide a simple method for recording income and expenses.
- To organise transactions using categories and dates.
- To calculate total income, total expenses and the remaining balance.
- To validate transaction details before storing them.
- To maintain transaction records in a suitable storage system.
- To handle invalid input and operational errors clearly.
- To apply basic secure coding practices while handling financial data.
- To develop a modular application that can be tested and maintained.

These objectives define what the project is intended to achieve. The report describes the proposed design and requirements; successful implementation and testing will be assessed separately as development progresses.

3.2 Project Scope

The initial scope is limited to personal expense management for a single user. The application will focus on essential transaction operations and straightforward financial summaries.

The planned scope includes:

- Recording income and expense transactions.
- Viewing stored transaction details.
- Assigning categories to transactions.
- Calculating income, expenses and balance.
- Validating user input.
- Updating or deleting records if included in the implemented version.
- Storing data for later retrieval.
- Testing the main functions and handling common errors.

The application is intended for personal use and learning purposes. It is not designed to replace professional accounting software or banking services.

3.3 Limitations

The initial version will have some limitations. It will not directly connect to bank accounts or automatically import transactions. It will also not process payments or transfer money.

Advanced features such as multiple user accounts, cloud synchronisation, mobile applications, automatic bank notifications and machine-learning-based spending predictions are outside the core scope.

The usefulness of the application will depend on users entering accurate transaction details. The system can organise and calculate the information provided, but it cannot independently verify whether a transaction actually occurred.

These limitations keep the project manageable and allow attention to remain on the core requirements.

4. System Architecture and Design

4.1 Overview of the Architecture

The proposed system follows a simple layered structure. Separating the application into layers will make the design easier to understand and maintain.

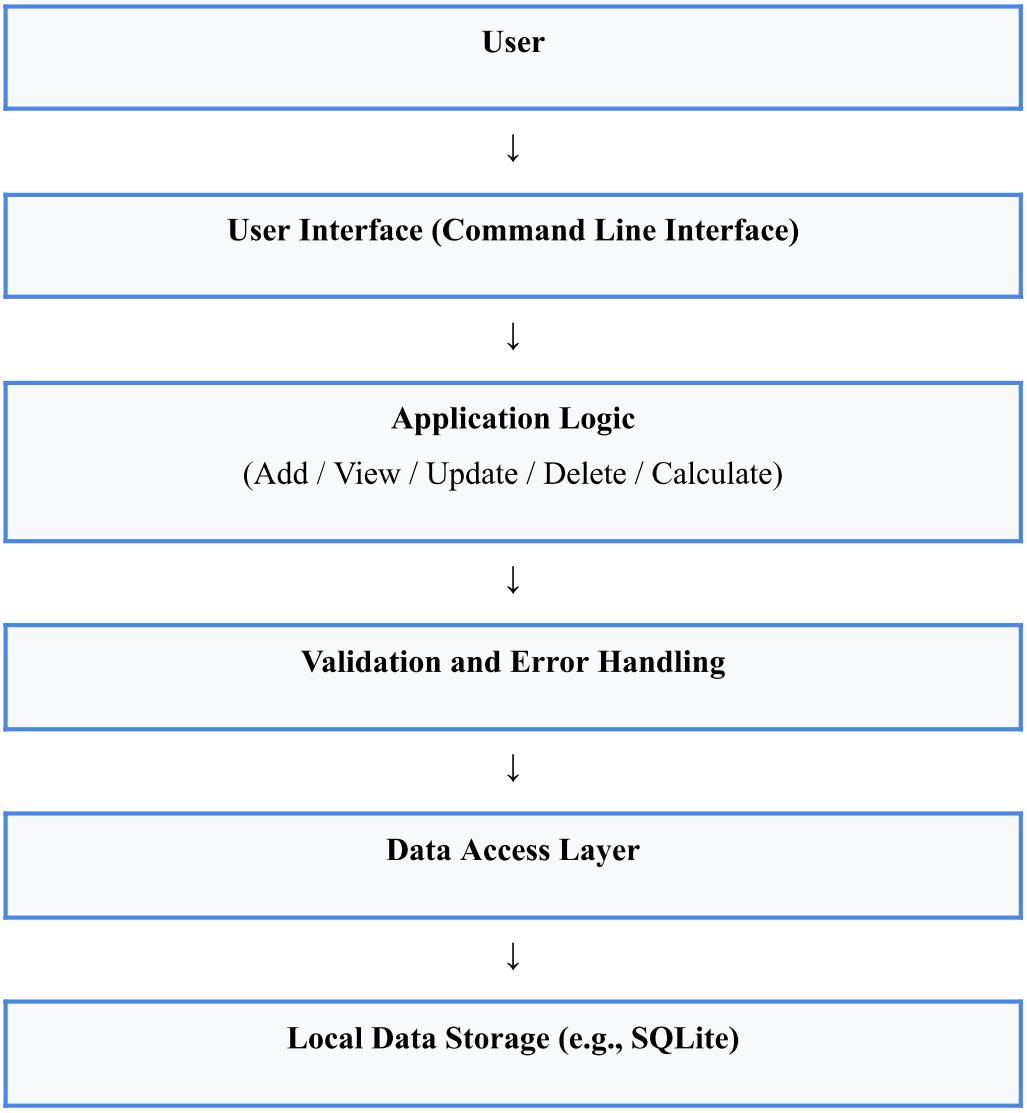
The user interacts with the interface to enter commands and view results. The application logic processes these requests, validates transaction details and performs calculations. The storage layer is responsible for saving and retrieving transaction records.

The main components are:

1. **User Interface:** Accepts user commands and displays information.
2. **Application Logic:** Processes requests and applies the expense-tracking rules.
3. **Validation and Error Handling:** Checks input and handles invalid operations.
4. **Data Access Layer:** Communicates with the storage system.
5. **Database or Local Storage:** Maintains transaction records.

The proposed architecture is shown below.

Figure 1: System Architecture



Source: Proposed design prepared for this project.

This architecture keeps user interaction separate from transaction processing and data storage. For example, the balance calculation can be handled by the application logic without requiring the user interface to perform database operations directly.

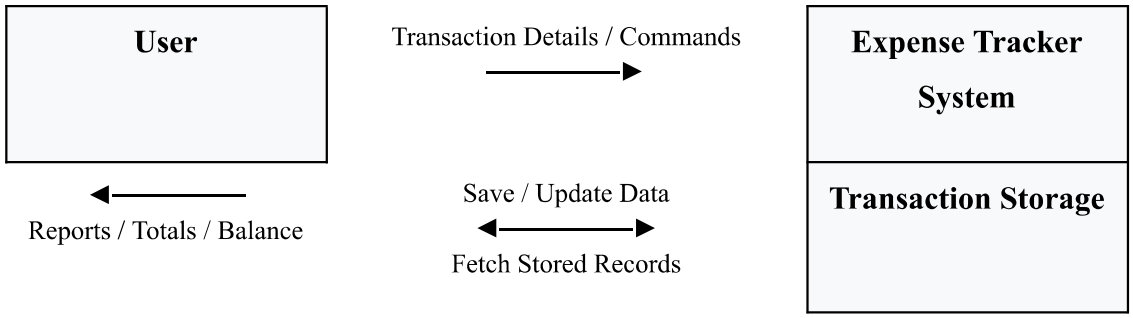
The separation also makes testing easier. Individual functions can be checked independently, while the complete application can be tested to verify that the components work together correctly.

4.2 Data Flow Diagram

A Data Flow Diagram (DFD) shows how information moves between the user, the application and the storage system.

In the proposed system, the user provides transaction details. The application validates the details and processes the requested operation. Valid records are saved to storage, while existing records can be retrieved to display transactions and calculate summaries.

Figure 2: Level 0 Data Flow Diagram



Source: Proposed data flow for the expense tracker.

The DFD illustrates the main exchange of information. The user initiates operations, the system processes them and the resulting information is returned to the user.

For an invalid transaction, the application should display an appropriate message rather than save incorrect data. Similarly, if storage is unavailable, the application should report the error instead of incorrectly indicating that the operation succeeded.

4.3 Proposed Data Design

Each transaction will require a small set of fields. The proposed structure is shown below.

Field	Purpose
Transaction ID	Uniquely identifies each record
Type	Indicates income or expense
Amount	Stores the transaction amount
Category	Groups transactions by purpose
Date	Records when the transaction occurred
Description	Provides optional additional details

The transaction ID will help the application identify a particular record when displaying, updating or deleting it.

The amount should be stored and processed carefully to avoid calculation errors. A suitable representation for monetary values will be selected during implementation, rather than relying on unrestricted floating-point calculations.

The transaction type must be checked before calculating totals. Income entries should contribute to total income, while expense entries should contribute to total expenses.

The remaining balance can be calculated using the following formula:

Remaining Balance = Total Income − Total Expenses

The balance represents the difference between the transactions recorded in the application. It should not be treated as a verified bank balance because the application does not connect to a bank account.

4.4 Basic Security Considerations

Although this is a small personal application, security should be considered from the design stage.

First, user input should be validated before processing. Invalid amounts, missing required fields and unsupported transaction types should be rejected with clear error messages.

Second, database operations should use appropriate parameterised queries when a relational database is selected. This helps reduce the risk of SQL injection.

Third, the application should avoid exposing sensitive financial information unnecessarily. If a local database is used, access to the device and the database file should be considered as part of the security design.

Finally, errors should be handled carefully. Error messages should help the user understand the problem without unnecessarily exposing internal technical details.

These measures provide a starting point for safer development. Additional controls will depend on the final implementation and the environment in which the application is used.

5. Project Timeline

The project will be developed in stages so that requirements, design, implementation and testing can be handled in an organised manner.

Stage	Planned Activities	Expected Outcome
Requirement Analysis	Identify the problem, users and essential features	Defined project requirements
System Design	Prepare the architecture, DFD and data structure	Documented system design
Core Development	Develop transaction entry and viewing functions	Basic transaction handling
Storage and Calculations	Add persistent storage and financial calculations	Stored records and summaries

Stage	Planned Activities	Expected Outcome
Testing and Correction	Test valid and invalid inputs and fix issues	More reliable application
Documentation	Prepare technical documentation and review the project	Completed project documentation

The stages provide a practical order for the work. Their duration can be adjusted according to development progress and the complexity of the selected storage method.

The design stage establishes the requirements and structure that will guide the implementation. Testing will focus on whether transactions are handled correctly, calculations produce expected results and errors are managed appropriately.

6. Expected Outcome

The expected outcome is a simple Go-based application that provides a structured way to record personal income and expenses.

Once implemented, the application is intended to allow a user to add transactions, view existing records, organise expenses by category and calculate total income, total expenses and the remaining balance.

The application should also provide clear feedback when a user enters invalid information or an operation fails. Persistent storage should make it possible to retrieve previously saved transactions in later sessions.

From a learning perspective, the project is expected to provide practical experience with Go programming, modular design, data storage, input validation, error handling and software testing.

These are expected outcomes rather than claims of completed functionality. The actual behaviour of the application will need to be verified through implementation and testing.

7. Conclusion and Future Scope

Phase I of the Smart Expense Tracker project establishes a strong foundation by defining the core requirements, system architecture, data flow, and technology stack. By selecting Go, the project leverages a efficient language suited for building modular and maintainable applications. The design prioritizes robust input validation, categorical expense tracking, and persistent storage.

In future phases, the scope can be expanded to include graphical user interfaces (GUI), multi-user support with secure authentication, multi-currency features, and automated analytical reporting.

8. References

[1] The Go Authors, “The Go Programming Language Documentation.” [Online]. Available: https://go.dev/doc/

[2] The Go Authors, “A Tour of Go.” [Online]. Available: https://go.dev/tour/

[3] SQLite, “SQLite Documentation.” [Online]. Available: https://www.sqlite.org/docs.html